EO/IR Detection Fusion Pipeline — Design & Plan

This document describes how to fuse the predictions of multiple per-sensor detectors (currently an **EO** model and an **IR** model, but designed to scale to *N* models) into a single, brightness-aware decision, and how to feed that decision into trajectory analysis.

It builds directly on parts that already exist in this repo:

Stage	Existing code
Mean-pixel brightness check	`brighntess/day_night.py`
Per-model sliced inference	`live/picture_inference_SAHI.py`

> **Scope note.** This is a **late (detection-level) fusion** pipeline, and **spatial alignment is intentionally skipped**. The EO and IR sensors are assumed to be co-boresighted (same field of view), so a detection at pixel `(x, y)` in one stream corresponds to roughly the same place in the other. Detections from both models therefore already live in a **shared image frame** and are compared directly — no homography / registration step. (The `fusion/` alignment scripts remain available if the hardware ever needs software co-registration, but the pipeline does not depend on them.)

---

1. Goal

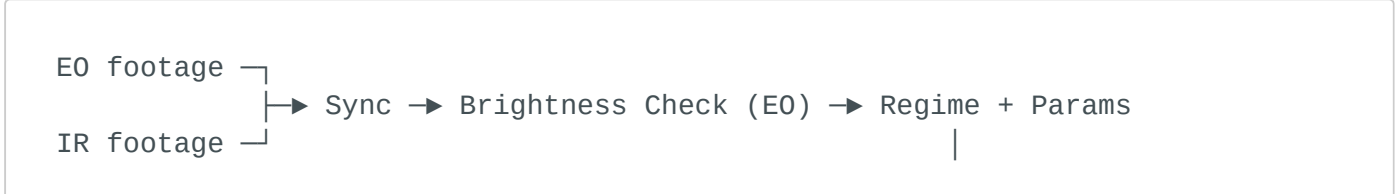
Two (or more) sensor streams look at the same scene. Each has a detector that outputs bounding boxes + confidence for the target object. We want **one** decided bounding box + a **single fused confidence** per frame, robust to lighting, and then to track that target over time.

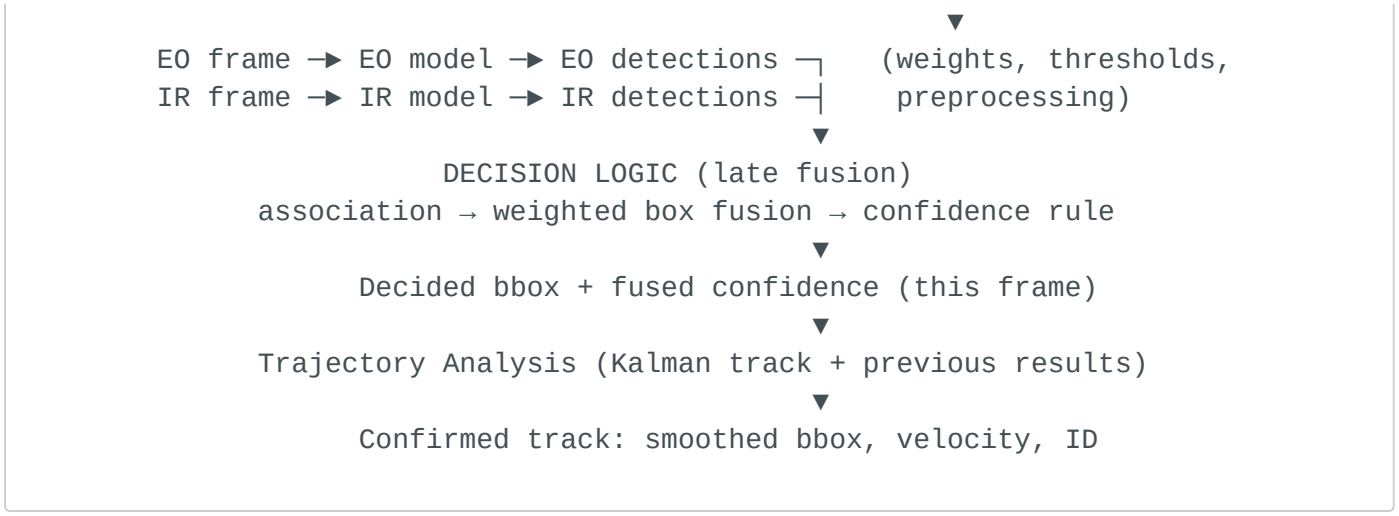
The core ideas:

1. **Lighting drives trust.** A brightness check on the EO stream tells us whether we are in a *day*, *twilight*, or *night* regime. EO is reliable in daylight; IR is reliable (and often better) in darkness. The regime sets how much we weight each sensor and what thresholds we use.
2. **Fuse at the detection level (late fusion).** Run each model on its own native imagery, then combine the resulting boxes — don't try to merge raw pixels into one image for a single detector. This keeps each model specialized to its modality and degrades gracefully if one sensor drops out.
3. **Confirm over time.** A single fused detection is a hypothesis; a smooth trajectory is evidence. A temporal filter (Kalman) both smooths the output and suppresses one-frame false positives.

---

2. Pipeline stages





> No spatial-registration stage: EO and IR detections are assumed co-boresighted (shared image frame), so they flow straight from the two models into the decision logic.

2.1 Sync

- **Temporal sync:** pair EO and IR frames by timestamp (nearest within a tolerance, e.g. ±½ frame interval). Fusion is meaningless if the two frames are from different moments. If one stream stalls, hold the last frame but flag it stale.
- **No spatial alignment.** The streams are assumed co-boresighted, so no homography or warping is applied. If their native resolutions differ, resize both to a common size so that pixel coordinates (and therefore IoU) are comparable; otherwise feed each model its native frame.

2.2 Brightness check → regime (EO only)

Reuse the logic in `day_night.py`: convert EO to HSV, take `mean(V)`. Instead of a binary day/night, bucket into three regimes with two thresholds `T_low`, `T_high`:

Regime	Condition	Meaning
`DAY`	`meanV ≥ T_high`	EO trustworthy
`TWILIGHT`	`T_low ≤ meanV < T_high`	both partially reliable
`NIGHT`	`meanV < T_low`	EO degraded, lean on IR

The regime selects a **parameter set** (next section). Optionally smooth `meanV` over a few frames (EMA) so a passing headlight or cloud doesn't flip the regime every frame.

> Note: brightness is computed on EO because that is what matches human-visible conditions. IR brightness reflects heat, not illumination, so it is not used for regime selection.

2.3 Per-model inference

- Run the EO model on the (optionally preprocessed) EO frame and the IR model on the IR frame. Use the SAHI sliced-inference pattern from `picture_inference_SAHI.py` if targets are small relative to the frame.
- Each detection is normalized to a common schema (§4).
- The active parameter set adjusts each model's **confidence threshold** and, in low light, EO **preprocessing** (e.g. CLAHE / gamma boost before inference).

2.4 Decision logic (the fusion core) — see §3.

2.5 Trajectory analysis — see §5.

> *(No spatial-registration stage — see the scope note above. EO and IR detections are taken as already living in a shared image frame, so they pass straight to §3.)*

---

3. Decision logic (fusion)

Input: a list of EO detections and a list of IR detections in a shared image frame (co-boresighted sensors — no registration applied), plus the active parameter set. Output: a small set of fused detections (often one) with a single confidence each.

Three sub-steps: **associate** → **fuse boxes** → **fuse confidence**.

3.1 Associate (which EO box = which IR box?)

- Compute IoU between every EO box and every IR box directly (they share a frame).
- Greedily match pairs with `IoU ≥ assoc_iou` (e.g. 0.4), highest IoU first.
- Three outcomes per object:

- **Agreement** — matched EO+IR pair (both sensors see it). Strongest evidence.
- **EO-only** — EO box with no IR match.
- **IR-only** — IR box with no EO match.

3.2 Fuse the box (Weighted Box Fusion)

For an agreement pair, the decided box is the **confidence-and-modality-weighted average** of the two corner sets — this is *Weighted Box Fusion* (WBF), which is more stable than NMS (NMS throws one box away; WBF averages them):

```
w_eo = modality_weight_EO(regime) * conf_eo
w_ir = modality_weight_IR(regime) * conf_ir
box  = (w_eo * box_eo + w_ir * box_ir) / (w_eo + w_ir)
```

For single-sensor detections the box is just that sensor's box.

3.3 Fuse the confidence

Let `a_EO`, `a_IR` be the per-regime modality weights (§ parameter table). Combine the weighted evidence with a **noisy-OR** so two independent agreeing sensors reinforce each other:

```
p_eo = a_EO * conf_eo          (0 if no EO detection)
p_ir = a_IR * conf_ir          (0 if no IR detection)
fused_conf = 1 - (1 - p_eo) * (1 - p_ir)
```

Then apply two adjustments:

- **Agreement bonus:** if both sensors fired and agreed spatially, multiply by `(1 + agreement_bonus)` (capped at 1.0). Independent confirmation is strong.
- **Single-sensor penalty in the wrong regime:** an EO-only detection at NIGHT, or an IR-only detection that EO *should* have seen in clear DAY, is discounted by `lonely_penalty`. This is where the regime really earns its keep.

Keep fused detections with `fused_conf ≥ decision_threshold`. For a single-target problem, output the highest-confidence survivor; for multi-target, output all.

3.4 Why these choices

- **Late / detection-level fusion** keeps each model native to its sensor and means a dead sensor just drops its contribution (graceful degradation).
- **WBF over NMS** because we *want* both sensors' geometry to inform the final box.
- **Noisy-OR** rewards independent agreement without letting one over-confident sensor dominate; weights and penalties bias it by lighting.

---

4. Common detection schema

Everything downstream of inference speaks one structure:

```
Detection = {
    "source":   "EO" | "IR",
    "bbox_xyxy": [x1, y1, x2, y2],   # shared-frame pixels (co-boresighted)
    "conf":      float,              # raw model confidence
    "class_id":  int,
    "frame_ts":  float,              # timestamp used for sync
}
```

Fused output:

```
FusedDetection = {
    "bbox_xyxy":  [x1, y1, x2, y2],
    "conf":       float,             # fused confidence
    "support":    ["EO", "IR"],      # which sensors backed it
    "regime":     "DAY"|"TWILIGHT"|"NIGHT",
}
```

---

5. Trajectory analysis

The fused per-frame detection is concatenated with previous results to form a track.

- **Filter:** a constant-velocity **Kalman filter** per target. State = `[cx, cy, w, h, vx, vy]`. Predict each frame, update with the fused box.

- **Association over time (gating):** the predicted next position defines a gate; a fused detection inside the gate updates the existing track, otherwise it seeds a new candidate track.
- **Confirm / delete:** a candidate becomes a confirmed track after *N* hits in *M* frames; a track is deleted after *K* misses. This is what removes one-frame false positives — a real target persists and moves smoothly.
- **Feedback (optional, powerful):** the predicted box can be fed back into the next frame's decision logic as a prior — boosting a fused detection that lands where the track expects it, and discounting one that appears from nowhere.
- **Output:** smoothed bbox, velocity vector, stable track ID, and track confidence.

---

6. Parameter table (tuned per regime)

These are the "parameters that get adjusted" in the diagram. Starting points to tune:

Parameter	DAY	TWILIGHT	NIGHT	Role
`modality_weight_EO`	0.70	0.50	0.30	trust in EO
`modality_weight_IR`	0.30	0.50	0.70	trust in IR
`conf_thresh_EO`	0.40	0.45	0.55	EO detections noisier in dark → raise
`conf_thresh_IR`	0.45	0.40	0.35	IR strong in dark → lower
`eo_preprocess`	none	gamma	CLAHE+gamma	help EO in low light
`assoc_iou`	0.40	0.40	0.40	EO/IR match threshold
`agreement_bonus`	0.15	0.20	0.20	reward independent confirmation
`lonely_penalty`	0.20	0.15	EO 0.40 / IR 0.10	discount unconfirmed singles

Parameter	DAY	TWILIGHT	NIGHT	Role
`decision_threshold`	0.45	0.45	0.45	keep fused detection

Brightness thresholds (`day_night.py` uses `140` on V): start `T_low ≈ 90`, `T_high ≈ 150`, then tune on your footage.

---

7. Reference pseudocode

```
def process_frame(eo_img, ir_img, tracks, params_by_regime):
    eo_img, ir_img = sync_and_size(eo_img, ir_img)      # §2.1 (no warping)

    regime = brightness_regime(eo_img)                   # §2.2 (reuse day_night)
    p = params_by_regime[regime]

    if p.eo_preprocess:
        eo_img = enhance(eo_img, p.eo_preprocess)

    eo_dets = run_model(EO_MODEL, eo_img, conf=p.conf_thresh_EO)   # §2.3
    ir_dets = run_model(IR_MODEL, ir_img, conf=p.conf_thresh_IR)

    # no spatial registration — co-boresighted, shared frame

    fused = decision_logic(eo_dets, ir_dets, p, regime)  # §3
    confirmed = trajectory_update(tracks, fused)         # §5
    return confirmed


def decision_logic(eo_dets, ir_dets, p, regime):
    pairs, eo_only, ir_only = associate(eo_dets, ir_dets, p.assoc_iou)
    out = []
    for e, i in pairs:                                   # agreement
        box  = wbf(e, i, p, regime)
        conf = noisy_or(p.modality_weight_EO * e.conf,
                        p.modality_weight_IR * i.conf)
        conf = min(1.0, conf * (1 + p.agreement_bonus))
        out.append(FusedDetection(box, conf, ["EO", "IR"], regime))
    for e in eo_only:
        conf = p.modality_weight_EO * e.conf * (1 - p.lonely_penalty_EO)
        out.append(FusedDetection(e.bbox, conf, ["EO"], regime))
    for i in ir_only:
        conf = p.modality_weight_IR * i.conf * (1 - p.lonely_penalty_IR)
        out.append(FusedDetection(i.bbox, conf, ["IR"], regime))
    return [d for d in out if d.conf >= p.decision_threshold]
```

---

8. Scaling beyond two models

The same structure handles *N* detectors: each contributes a detection list and a per-regime weight. Association becomes clustering of mutually-overlapping boxes across all sources; WBF averages each cluster; noisy-OR generalizes to `1 - Π(1 - a_s·conf_s)` over all sources `s`. Nothing in the design assumes exactly two sensors — only that each has a weight that may depend on the regime.

---

9. Failure / edge-case handling

Situation	Behavior
One sensor stream drops	Fuse from the surviving sensor only; flag reduced confidence.
Frames out of sync	Skip fusion or widen gate; never fuse mismatched timestamps.
Boresight drift (cameras shift apart)	Agreeing detections stop overlapping → association rate falls; treat as the signal to re-check sensor alignment. With alignment skipped, a persistent EO/IR offset will read as disagreement.
EO blinded (glare/headlights)	High `meanV` variance; trust IR, raise EO threshold.
Disagreement (boxes don't overlap)	Keep both as singles with `lonely_penalty`; let trajectory analysis decide which persists.

---

10. Implementation roadmap

1. **Wrap inference** — turn `picture_inference_SAHI.py` into a reusable `run_model(model, img, conf) -> [Detection]` for both EO and IR weights.
2. **Regime module** — generalize `day_night.py` to return `DAY/TWILIGHT/NIGHT` + the parameter set, with EMA smoothing.

3. **Decision logic** — implement §3 (`associate`, `wbf`, `noisy_or`). No registration module is needed (co-boresighted assumption).
4. **Trajectory** — constant-velocity Kalman + gating/confirm/delete (§5).
5. **Tune** thresholds in §6 on labeled day *and* night clips; measure fused vs. single-sensor precision/recall to prove the fusion actually helps.